ShanghaiTech University

CS101 Algorithms and Data Structures
Fall 2026

Homework 3

Due date: October 18, 2026, at 23:59

1. Please write your solutions in English.
2. You may discuss the assignment with classmates and AI tools, but you must not directly copy answers or adapt AI-generated contents in any form.
3. Set your FULL name to your Chinese name and your STUDENT ID correctly in Gradescope account settings.
4. Submit your solutions to Gradescope. When submitting, match your solutions to the problems correctly.
5. We recommend that you write your homework in LaTeX. If you want to submit a handwritten version, scan it clearly. `CamScanner` is recommended.
6. Submit your homework on time. No late submission will be accepted.
7. Violations of any of the above may result in zero points.

Important Notes:

1. Some problems in this homework require you to design a divide-and-conquer algorithm. When grading these problems, we will put more emphasis on how you reduce a problem to a smaller problem and how to combine their solutions with the divide-and-conquer strategy.

2. There are mainly 3 kinds of writing problems that may be involved in this assignment (and following assignments). We recommend that you know what each part **SHOULD** include:

    1. Describe/Design an algorithm that...: A clear description of your algorithm design in natural language is needed. If you want to present it in pseudocode, you'd better take it seriously on your boundary cases and supplement it with meaningful, clear variable names. Unless the problem allows, you should give a complete algorithm instead of fragmented words, incrementally developed by some other algorithm.
    2. Justify your algorithm/Justify its correctness/...: You should give a proof of why your algorithm is correct. The requirements may not be so strict, but please at least clarify every step of your proof and make sure they are reasonable. You can give out an answer or some propositions and then make **reliable and credible** explanations, and we encourage you to do that in suitable cases (e.g., using induction).
    3. Analyze the time complexity/Show its time complexity is ...: You should give out an exact time complexity and some reason to justify it (e.g,. Master Theorem/Recurrence relation/...). We won't restrict too much on O and Θ. However, your time complexity bound should be **tight**. For example, we won't make it correct when you say something is $O(n^2)$ with its worst case running on $\Theta(n \log n)$ (Even if it is logically correct).

3. Your answer for these problems is **not allowed to include real C or C++ code**.

4. In your description of algorithm design, you should describe each step of your algorithm clearly.

5. You are encouraged to write pseudocode to facilitate explaining your algorithm design, though this is not mandatory. However, we recommend that you not write pseudocode if you are using handwriting to complete your homework. If you choose to write pseudocode, please give some additional descriptions to make your pseudocode intelligible.

6. You are recommended to finish the algorithm design part of this homework with LaTeX. If you insist on using handwriting to finish it, your explanation should be at least tidy and readable (can be specified); otherwise, your assignments will be considered as unspecified.

7. Please **DO NOT** just hand in a picture of the answer sheet with several answers only. For both of your and our convenience, please at least upload a PDF assignment file (In fact, you don't need to cut down any pages because we will filter it for you).

0. (0 Points) Binary Search Example

Design an algorithm to find the index of an element x in a sorted array a of n elements.

Solution: Algorithm Design: We basically ignore half of the elements just after one comparison.

1. Compare x with the middle element.
2. If x matches the middle element, return the middle index.
3. Otherwise x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
4. Otherwise (x is smaller) recur for the left half.

Pseudocode(Optional):

```
1:  function BINARYSEARCH(a, value, left, right)
2:      if right < left then
3:          return not found
4:      end if
5:      mid ← ⌊(right − left)/2⌋ + left
6:      if a[mid] = value then
7:          return mid
8:      end if
9:      if value < a[mid] then
10:         return binarySearch(a, value, left, mid-1)
11:     else
12:         return binarySearch(a, value, mid+1, right)
13:     end if
14: end function
```

Proof of Correctness: If x happens to be the middle element, we will find it in the first step. Otherwise, if x is greater than the middle element, then all the element in the left half subarray is less than x since the original array has already been sorted, so we just need to look for x in the right half subarray. Similarly, if x is less than the middle element, then all the element in the right subarray is greater than x, so we just need to look for x in the front list. If we still can't find x in a recursive call where $left = right$, which indicates that x is not in a, we will return $not\ found$ in the next recursive call.

Time Complexity Analysis: During each recursion, the calculation of mid and comparison can be done in constant time, which is $O(1)$. We ignore half of the elements after

each comparison; thus we need $O(\log n)$ recursions.

$$T(n) = T(n/2) + O(1)$$

Therefore, by the Master Theorem $\log_b a = 0 = d$, so $T(n) = O(\log n)$.

1. (15 points) Multiple Choices

Each question has **one or more** correct answer(s). Select all the correct answer(s). For each question, you will get 0 points if you select one or more wrong answers, but you will get 1 point if you select a non-empty subset of the correct answers.

Write your answers in the following table.

(a)	(b)	(c)	(d)	(e)

(a) (3') Which of the following sorting algorithm(s) is(are) stable?

A. Bubble-Sort

B. Quick-Sort (picking the first element as pivot)

C. Insertion-Sort

D. Merge-Sort

(b) (3') Given n distinct elements

$$a_1 < a_2 < \cdots < a_n,$$

suppose Randomized Quick-Sort chooses the pivot uniformly at random from the current subarray. Which of the following statement(s) is(are) **true**?

A. Any two adjacent elements a_i and a_{i+1} will be compared during the execution of Randomized Quick-Sort.

B. For $i < j$, the probability that a_i and a_j are compared is

$$\frac{2}{j-i+1}.$$

C. The probability that a_1 and a_n are compared is

$$\frac{2}{n}.$$

D. The expected number of comparisons made by Randomized Quick-Sort is $\Theta(n^2)$.

(c) (3') Which of the following statement(s) about Quick-Sort is(are) **true**?

A. The median-of-three strategy chooses the median of the first, middle, and last entries as the pivot. It may give a better approximation of the true median, but it does not guarantee a balanced partition.

B. If Quick-Sort uses in-place partitioning, then it uses $\Theta(1)$ extra memory in every case.

C. For sufficiently small subarrays, using Insertion-Sort instead of continuing the Quick-Sort recursion can be a reasonable optimization.

D. The recursion depth of Quick-Sort is always $\Theta(\log n)$, regardless of how the pivots are chosen.

(d) (3') Which of the following statement(s) about divide-and-conquer algorithms and their applications is(are) **true**?

A. A divide-and-conquer algorithm usually consists of three steps: dividing the original problem into smaller subproblems, recursively solving the subproblems, and combining their solutions.

B. Any recursive algorithm can be considered a divide-and-conquer algorithm because it always solves smaller instances of the original problem.

AC

C. FFT applies the divide-and-conquer idea by recursively decomposing a polynomial into its even-degree and odd-degree parts.

D. If a divide-and-conquer algorithm divides a problem of size n into two subproblems of size $n/2$, its running time must be $\Theta(n \log n)$, regardless of the cost of combining the solutions.

(e) (3') Which of the following statement(s) about divide-and-conquer sorting algorithms is(are) **true**?

A. In merge sort, if every element in the left half is smaller than every element in the right half, the recursive sorting of both halves can be skipped.

B. After a correct quicksort partition, every element in the left partition is no greater than any element in the right partition, even though neither partition is necessarily sorted.

BC

C. When merging two sorted subarrays of lengths p and q, the standard merge procedure requires at most $p + q - 1$ element comparisons.

D. If quicksort consistently divides a problem of size n into subproblems of approximately $n/4$ and $3n/4$, its running time is $\Theta(n^2)$ because the partitions are unbalanced.

2. (8 points) Which Sort?

Given a sequence

$$A = \langle 24, 7, 18, 3, 2026, 11, 5, 18, 2, 16 \rangle,$$

we have performed some different sorting algorithms on it, during which some intermediate results are printed. Note that the steps listed below may **not** appear consecutively in the algorithm, but they are presented in the correct logical order.

For each group of steps, guess ($\surd$) what the algorithm is. The algorithm might be one among the following choices:

- Insertion-sort, implemented in the way that avoids swapping elements
- Bubble-sort, which stops immediately when no swap happens during one iteration
- Merge-sort
- Quick-sort, with pivot chosen to be A_l when partitioning a subarray $A_l, \cdots, A_r$.

(a) (2')

$\langle 7, 18, 24, 3, 2026 \mid 11, 5, 18, 2, 16 \rangle$,
$\langle 3, 7, 18, 24, 2026 \mid 11, 5, 18, 2, 16 \rangle$,
$\langle 3, 7, 18, 24, 2026 \mid 5, 11, 18, 2, 16 \rangle$,
$\langle 3, 7, 18, 24, 2026 \mid 2, 5, 11, 16, 18 \rangle$.

◯ Insertion-sort ◯ Bubble-sort ✓ Merge-sort ◯ Quick-sort

(b) (2')

$\langle 7, 18, 3, 24, 11, 5, 18, 2, 16, 2026 \rangle$,
$\langle 7, 3, 18, 11, 5, 18, 2, 16, 24, 2026 \rangle$,
$\langle 3, 7, 5, 11, 2, 16, 18, 18, 24, 2026 \rangle$,
$\langle 3, 2, 5, 7, 11, 16, 18, 18, 24, 2026 \rangle$.

◯ Insertion-sort ✓ Bubble-sort ◯ Merge-sort ◯ Quick-sort

(c) (2')

$\langle 16, 7, 18, 3, 11, 5, 18, 2, 24, 2026 \rangle$,
$\langle 2, 7, 3, 11, 5, 16, 18, 18, 24, 2026 \rangle$,
$\langle 2, 5, 3, 7, 11, 16, 18, 18, 24, 2026 \rangle$,
$\langle 2, 3, 5, 7, 11, 16, 18, 18, 24, 2026 \rangle$.

◯ Insertion-sort ◯ Bubble-sort ◯ Merge-sort ✓ Quick-sort

(d) (2')

$\langle 7, 18, 24, 3, 2026, 11, 5, 18, 2, 16 \rangle$,
$\langle 3, 7, 18, 24, 2026, 11, 5, 18, 2, 16 \rangle$,
$\langle 3, 5, 7, 11, 18, 24, 2026, 18, 2, 16 \rangle$,
$\langle 2, 3, 5, 7, 11, 18, 18, 24, 2026, 16 \rangle$.

✓ Insertion-sort ◯ Bubble-sort ◯ Merge-sort ◯ Quick-sort

3. (17 points) Master Theorem

Assume that n is a power of 2, and every basic operation other than a recursive call takes $\Theta(1)$ time.

Consider the following procedure:

```
void Mystery(int n){
    if(n <= 1) return;

    for(int r = 0; r < 3; r++){
        Mystery(n / 2);
    }

    for(int i = 0; i < n; i++){
        for(int j = 0; j < floor(sqrt(n)); j++){
            // perform a constant-time operation
        }
    }
}
```

Give tight asymptotic bounds in $\Theta(\cdot)$ notation.

(a) (3') Complete the recurrence for the running time of `Mystery`:

$$T(n) = \underbrace{3}_{a} T\left(\frac{n}{\underbrace{2}_{b}}\right) + \Theta\left(\underbrace{n\sqrt{n}}_{f(n)}\right).$$

Solution:

In Recursion, the program produces three $T(\frac{n}{2})$ sub question

In the for loop the Runtime is $\Theta(n \cdot \sqrt{n})$. with $\Theta(n)$ times in outer layer and $O(\sqrt{n})$ in inner layer.

$a = 3$, $b = 2$

$f(n) = n\sqrt{n}$

(b) (4') Apply the Master Theorem to the recurrence in part (a). State the applicable case and give a tight asymptotic bound for $T(n)$.

Solution:

$p = \log_b a = \log_2 3 > n^{\frac{3}{2}}$.

$\therefore T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 3})$

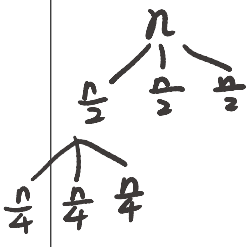
(c) (4') Use a recursion tree to analyze the recurrence in part (a).

i. At level i, find the number of subproblems, the size of each subproblem, and the total number W_i of constant-time operations performed by the nested loops.

ii. Find the depth of the recursion tree and determine whether W_i increases, decreases, or remains asymptotically unchanged as i increases.

iii. Derive a tight asymptotic bound for $T(n)$ from the recursion tree.

Solution:



i: In level, the number of subproblems is 3^i

the size of each subproblem: $\frac{n}{2^i}$

$W_i = 3^i \times (\frac{n}{2^i})^{\frac{3}{2}} = n^{\frac{3}{2}} \times (\frac{3}{2^{1.5}})^i$

∵ depth: $\Theta(\log_2 n)$

When i increases W_i increases

iii $W_{\log_2 n} = n^{1.5} \times (\frac{3}{2^{1.5}})^{\log_2 n} = n^{1.5} \times \frac{n^{\log_2 3}}{n^{1.5}} = n^{\log_2 3}$

$T(n) = \sum W_i = \Theta(n^{\log_2 3})$

(d) (4') For $p > 0$, analyze

$$T_p(n) = 4T_p(n/2) + \Theta(n^p).$$

Give the Master Theorem case and the tight asymptotic bound for each possible range of p.

Solution:

$b=4$, $a=\frac{1}{2}$.

$p=\log_4 \frac{1}{2} = -0.5$.

① When $P > -0.5$

then $T_p(n) = \Theta(n^p)$

② When $P < -0.5$. $T_p(n) = \Theta(n^{-0.5})$.

③ When $P = -0.5$ $T_p(n) = \Theta($ $)$

(e) (2') Consider the recurrence

$$R(n) = R(n/2) + R(n/4) + \Theta(n^{3/2}).$$

Can the Master Theorem be applied directly to this recurrence? If not, use appropriate upper and lower bounds together with the Master Theorem to determine a tight asymptotic bound for $R(n)$.

Solution:

upper bound:

$R(n) \le R(\frac{n}{2}) + R(\frac{n}{4}) + \Theta(n^{\frac{3}{2}}) = 2R(\frac{n}{2}) + \Theta(n^{\frac{3}{2}})$

$p = \log_2 2 = 1 < \frac{3}{2}$

$\therefore R(n) = \Omega(n^{\frac{3}{2}})$

lower bound:

$R(n) \ge 2R(\frac{n}{4}) + \Theta(n^{\frac{3}{2}})$

$p = \log_4 2 = 0.5 < \frac{3}{2}$, $R(n) = O(n^{\frac{3}{2}})$

$\Omega(n^{\frac{3}{2}}) \le R(n) \le O(n^{\frac{3}{2}})$.

$R(n) = \Theta(n^{\frac{3}{2}})$.

4. (15 points) Quick-sort Partitioning and Tail-Recursion Elimination

In Quick-sort, to perform space-efficient *in-place* partitioning on a subarray $[l, r]$, two standard schemes are commonly adopted:

- **Lomuto partition** (often chooses the right end as pivot). It scans once, maintaining a boundary of "$\leq$ pivot" and finally swaps the pivot into position.
- **Hoare partition** (often chooses the left end as pivot). Two indices move inward: the left one stops at "$\geq$ pivot", the right one stops at "$\leq$ pivot"; if left $\leq$ right, swap and continue; otherwise return the index.

(a) (4')

```
int LomutoPartition(std::vector<int>& A, int l, int r) {
    int pivot = A[r];
    int i = l - 1;
    for (int j = l; j <= r - 1; ++j) {
        if (A[j] <= pivot) {
            ++i;
            std::swap(A[i], A[j]);
        }
    }
    std::swap(A[i + 1], A[r]);
    return i + 1;
}
```

Apply *Lomuto*'s rule as described above. Given

$$A = \langle 9, 4, 8, 2, 7, 1, 5, 6 \rangle,$$

perform exactly one partition on the **0-based** subarray $[1, 6]$, i.e. LomutoPartition(A,1,6).

1. The return is: 4. (2 pts)
2. The full array A after **this single partition** (write $A[0]$ to $A[7]$): $\langle 9\ 4\ 2\ 1\ 5\ 8\ 7\ 6 \rangle$. (2 pts)

(b) (4') Complete the following C++ skeleton. Use Hoare partition as defined previously (**chooses the left end as pivot**). All indices are **0-based**, range $[l, r]$ inclusive.

```
int hoare_partition(std::vector<int>& a, int l, int r) {
    int pivot = a[l];
    int i = l - 1, j = r + 1;
    while (true) {
        i++;
        while (i <= r && a[i] <= pivot)
            i++;
        j--;
        while (j >= l && a[j] >= pivot)
            j--;
        if (i >= j)
            return j;
```

```
            std::swap(a[i], a[j]);
        }
    }
    void QuickSort(std::vector<int>& a, int l, int r) {
        if (l < r) {
            int p = hoare_partition(a, l, r);
            QuickSort(a, l, p);
            QuickSort(a, p + 1, r);
        }
    }
```

Solution:

a[l]
a[i] < pivot
a[j] > pivot
std::swap(a[i], a[j])

(c) (4') Tail recursion is a special form of recursion where the recursive call is the final operation in the function. This means that there is no additional work to be done after the recursive call returns. The key advantage of tail recursion is that it can be optimized by the compiler, thereby reducing the overhead of multiple function calls and stack frames.

In the case of Quick-sort, we can apply tail recursion optimization by choosing to recurse on the smaller subarray first and then iterating over the larger subarray. This reduces the recursion depth to $O(\log n)$ instead of $O(n)$ in the worst case.

Complete the following C++ skeleton using tail recursion optimization.

```
void QuickSort(std::vector<int>& a, int l, int r) {
    while (l < r) {
        int p = hoare_partition(a, l, r);
        if ((p - l) < (r - p)) {
            QuickSort(a, l, p);
            _________;
        } else {
            QuickSort(a, p + 1, r);
            _________;
        }
    }
}
```

Solution:

QuickSort(a, p+1, r)
Quicksort (a, l, p)

(d) (3') **Multiple choice**

Which statement(s) is(are) **True** about Quick-sort?

A. Tail recursion optimization improves the worst-case time complexity of Quick-sort to $O(n \log n)$.

B. The Lomuto partition scheme guarantees that the pivot ends up at its final position.

AB

C. The Hoare partition scheme guarantees that the worst-case time complexity will be $O(n \log n)$.

D. The Hoare partition scheme always performs fewer swaps than Lomuto and returns the pivot(the left end)'s final index.

5. (10 points) Karatsuba's Algorithm

Let A, B are two polynomials where $A(x) = \sum_{i=0}^{n} a_i x^i, B(x) = \sum_{i=0}^{n} b_i x^i$.

The goal of this problem is to invent an algorithm to multiply two polynomials faster.

(a) (1') The time complexity of calculating the product of two polynomials of n or less degree trivially (i.e. use $C(x) = \sum_{0 \le i,j \le n} a_i b_j x^{i+j}$) is $O(n^2)$.

(b) (3') Recall the vertical multiplication on polynomials. Rewrite $A(x) = A_1(x)x^{\frac{n}{2}} + A_2(x)$ and $B(x) = B_1(x)x^{\frac{n}{2}} + B_2(x)$, where A_i, B_i are polynomials of $\frac{n}{2}$ or less degree :

$$\begin{array}{lll} & A_1 x^{\frac{n}{2}} & +A_2 \\ & \times B_1 x^{\frac{n}{2}} & +B_2 \\ \hline & A_1 B_2 x^{\frac{n}{2}} & +A_2 B_2 \\ A_1 B_1 x^n & +A_2 B_1 x^{\frac{n}{2}} & \\ \hline A_1 B_1 x^n & +(A_1 B_2 + A_2 B_1) x^{\frac{n}{2}} & +A_2 B_2. \end{array}$$

Write down the recurrence relation of this algorithm's time complexity and calculate it:

Solution:

$T(n) = 4T(\frac{n}{2}) + \Theta(n)$

$p = \log_2 4 = 2 > 1$

$\therefore T(n) = \Theta(n^2)$

(c) (3') Recall how Strassen's algorithm works for matrix multiplication: It decreases one time of multiplication (from 8 to 7), then its time complexity turns into $\Theta(\underline{\quad n^{\log_2 7} \quad})$ from $\Theta(n^3)$.

Consider how to reduce one multiplication, we wonder if $A_1B_2 + A_2B_1$ can be calculated with 1 multiplication with proper polynomial calculation:

Hint: You can use any number of additions. Think out reuse A_1B_1 and A_2B_2. (A_1B_1 and A_2B_2 won't be considered as extra multiplication.)

$A_1B_2 + A_2B_1 = \underline{(A_1 - A_2) \cdot (B_2 - B_1) + A_1B_1 + A_2B_2}$ $(A_1 - A_2)(B_2 - B_1)$

(d) (3') After the modification, we obtain a method to use 1 less multiplication to calculate the polynomial multiplication.

Write down the recurrence relation of the modified algorithm's time complexity and calculate it.

Solution:

$T(n) = 3T(\frac{n}{2}) + \Theta(n)$

$\log_2 3 > 1$

$\therefore T(n) = \Theta(n^{\log_2 3})$

6. (18 points) TA's Letter Board

During a CS101 recitation, the TAs prepare a small game using magnetic tiles. All tiles are placed on a narrow rail, so the only allowed operation is to swap two **adjacent** tiles.

Each tile has a distinct label from 1 to n.

The TAs want to transform the current arrangement

$$A = \langle a_1, a_2, \ldots, a_n \rangle$$

into a target arrangement

$$B = \langle b_1, b_2, \ldots, b_n \rangle,$$

where both A and B are permutations of $\{1, 2, \ldots, n\}$.

For example,

$$\langle 1, 2, 3 \rangle \to \langle 2, 1, 3 \rangle \to \langle 2, 3, 1 \rangle$$

uses two adjacent swaps.

The TAs first try the obvious simulation, but soon realize that n may be very large. Your task is to help them compute the minimum number of adjacent swaps efficiently.

I. We first convert the tile arrangement into an integer sequence.

For each tile x, let $\mathrm{pos}_B(x)$ denote its position in B. Then construct the sequence

$$P = \langle p_1, p_2, \ldots, p_n \rangle, \qquad p_i = \mathrm{pos}_B(a_i).$$

(a) (2') Let

$$A = \langle 1, 2, 3, 4, 5 \rangle, \qquad B = \langle 4, 1, 3, 5, 2 \rangle.$$

Construct the corresponding integer sequence

$$P = \langle p_1, p_2, \ldots, p_n \rangle.$$

Solution:

$p_1 = 2,\ p_2 = 5,\ p_3 = 3,\ p_4 = 1,\ p_5 = 4.$

$P = \langle 2, 5, 3, 1, 4 \rangle$

(b) (3') For the example from part (a),

$$P = \langle 2, 5, 3, 1, 4 \rangle,$$

count the number of inversions in P and list all the inversions.

Solution:
<5,3>, <5,1>, <5,4>, <3,1>, <2,1>

total: 5 inversions

(c) (10') Design a divide-and-conquer algorithm to compute the minimum number of adjacent swaps needed to transform A into B.

Your algorithm should use the sequence P defined above and should run faster than $\Theta(n^2)$.

Explain why your algorithm is correct.

Hint: Recall merge sort and inversions.

Solution:
For P[l···r], mid = $\lfloor \frac{l+r}{2} \rfloor$, divide into P[l,···mid] and P[mid+1,···r]
Using recursion to calculate inversions in the left and right part, and sort them
Merge two already-sorted array., calculate inversions with one in left sequence, one in right sequence
For this, use double pointer, left pointer: i, right pointer: j
if P[i] ≤ P[j], put P[i] into a temporary array, i moves back
if P[i] > P[j]: # of inversions will add by (mid-i+1).
then put P[j] in the temporary array, j moves back

正确性请帮我简要回答.

Solution:

Continue:

(d) (3') Analyze the time complexity of the complete algorithm.

Solution:

Write the recursion of $T(n)$

$T(n) = 2T(\frac{n}{2}) + O(n)$

$\log_2 2 = 1$, Using master theorem

$\therefore T(n) = O(n \log n)$

7. (17 points) TA's Grade Grid

After grading a CS101 assignment, the TAs store the scores in an $n \times n$ grid

$$A = (a_{ij}),$$

where n is a positive integer.

Each row has a unique minimum element. For row i, let

$$m_i = \arg\min_{1 \le j \le n} a_{ij}$$

be the column containing its minimum.

The grading system guarantees that the positions of the row minima are nondecreasing:

$$m_1 \le m_2 \le \cdots \le m_n.$$

For example, consider the following 4×4 grid:

最小元素位置由上至下递增

$$A = \begin{pmatrix} 12 & 5 & 8 & 10 \\ 9 & 2 & 7 & 6 \\ 8 & 7 & 3 & 9 \\ 11 & 8 & 6 & 1 \end{pmatrix}.$$

The minimum elements of the four rows are 5, 2, 3, and 1, respectively. Their column positions are

$$\langle m_1, m_2, m_3, m_4 \rangle = \langle 2, 2, 3, 4 \rangle,$$

which satisfy the nondecreasing condition.

Your goal is to find

$$m_1, m_2, \ldots, m_n.$$

Design a divide-and-conquer algorithm that runs in $o(n^2)$ time.

Solution:

① We first find the middle row of the grid. mid = $\frac{\text{row_low} + \text{row_high}}{2}$

② for i = mid, We find the smallest number and get its position m_{mid}, which costs O(n)

③ For the upper section, and lower section, recursively doing ① and ②

The base case is: row_low ≥ row_high

$T(n) = O(\log n) \times O(n) = O(n \log n)$

↓ the depth of recursion tree

↓ costs in every level

Solution: